Pertemuan 7
Aljabar Linier Komputasional

Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008 Kelas RA dan RB

Triyana Muliawati, S.Si., M.Si.
Rifky Fauzi

Program Studi Matematika
Institut Teknologi Sumatera

Semester Ganjil 2026/2027

Hari ini

- Vektor dan matriks sebagai data.
- Operasi baris–kolom sebagai algoritma.
- Verifikasi numerik: residual dan norma.
- Invers, SPL, eigenvalue, dan PCA sebagai komputasi.
- Fungsi kerja dari kode, lalu orde pertumbuhannya.

Aljabar linear dari sisi komputasi

Objek matematika

- Vektor: daftar komponen.
- Matriks: tabel dua dimensi.
- Transformasi linear: operasi terhadap vektor.
- SPL: mencari x pada $Ax = b$.
- Eigen: arah yang hanya berubah skala.

Sudut pandang program

- Bagaimana indeks dibaca?
- Loop mana yang diperlukan?
- Apakah dimensi cocok?
- Berapa banyak kerja ketika ukuran data membesar?
- Bagaimana memeriksa jawaban numerik?

Prinsip kerja

Rumus linear algebra tidak hilang. Kita menerjemahkannya menjadi struktur data, indeks, algoritma, dan pemeriksaan hasil.

Vektor: indeks, operasi komponen, dan norma

Dari rumus ke indeks

$$x = (x_1, \ldots, x_n), \qquad y = (y_1, \ldots, y_n)$$

$$x + y = (x_i + y_i)_{i=1}^{n}, \quad x \cdot y = \sum_{i=1}^{n} x_i y_i$$

$$\|x - y\|_2 = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Vektor: indeks, operasi komponen, dan norma

Dari rumus ke indeks

$$x = (x_1, \ldots, x_n), \qquad y = (y_1, \ldots, y_n)$$

$$x + y = (x_i + y_i)_{i=1}^{n}, \quad x \cdot y = \sum_{i=1}^{n} x_i y_i$$

$$\|x - y\|_2 = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Fungsi kerja

Jika loop berjalan pada n komponen, maka banyak kerja bertambah sebanding dengan n.

$$T(n) = \text{kerja tetap} + \text{kerja per komponen} \cdot n$$

$$T(n) \in O(n)$$

```python
def dot(x, y):
    if len(x) != len(y):
        raise ValueError("dimensi beda")
    total = 0
    for i in range(len(x)):
        total += x[i] * y[i]
    return total

def norm_diff(x, y):
    return dot([x[i]-y[i] for i in range(len(x))],
               [x[i]-y[i] for i in range(len(x))])
                   ** 0.5
```

Element-wise dan perkalian matriks

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 2 & 0 \\ 1 & 5 \end{pmatrix}$$

$$A \odot B = \begin{pmatrix} 1 \cdot 2 & 2 \cdot 0 \\ 3 \cdot 1 & 4 \cdot 5 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 3 & 20 \end{pmatrix}$$

$$AB = \begin{pmatrix} 1 \cdot 2 + 2 \cdot 1 & 1 \cdot 0 + 2 \cdot 5 \\ 3 \cdot 2 + 4 \cdot 1 & 3 \cdot 0 + 4 \cdot 5 \end{pmatrix} = \begin{pmatrix} 4 & 10 \\ 10 & 20 \end{pmatrix}$$

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[2, 0], [1, 5]])

A * B # element-wise
A @ B # matrix product
```

Element-wise dan perkalian matriks

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 2 & 0 \\ 1 & 5 \end{pmatrix}$$

$$A \odot B = \begin{pmatrix} 1 \cdot 2 & 2 \cdot 0 \\ 3 \cdot 1 & 4 \cdot 5 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 3 & 20 \end{pmatrix}$$

$$AB = \begin{pmatrix} 1 \cdot 2 + 2 \cdot 1 & 1 \cdot 0 + 2 \cdot 5 \\ 3 \cdot 2 + 4 \cdot 1 & 3 \cdot 0 + 4 \cdot 5 \end{pmatrix} = \begin{pmatrix} 4 & 10 \\ 10 & 20 \end{pmatrix}$$

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[2, 0], [1, 5]])

A * B # element-wise
A @ B # matrix product
```

Catatan

`*` membaca posisi yang sama. `@` membaca baris pada A dan kolom pada B.

Perkalian matriks: pseudocode dulu, kode kemudian

Pseudocode

```
INPUT: A berukuran m x p, B berukuran p x n
OUTPUT: C = A B berukuran m x n

cek banyak kolom A = banyak baris B
buat C berisi nol, ukuran m x n
FOR i = 0 sampai m-1:
  FOR j = 0 sampai n-1:
    s = 0
    FOR k = 0 sampai p-1:
      s = s + A[i][k] * B[k][j]
    C[i][j] = s
RETURN C
```

Perkalian matriks: pseudocode dulu, kode kemudian

Pseudocode

```
INPUT: A berukuran m x p, B berukuran p x n
OUTPUT: C = A B berukuran m x n

cek banyak kolom A = banyak baris B
buat C berisi nol, ukuran m x n
FOR i = 0 sampai m-1:
  FOR j = 0 sampai n-1:
    s = 0
    FOR k = 0 sampai p-1:
      s = s + A[i][k] * B[k][j]
    C[i][j] = s
RETURN C
```

```python
def matmul_manual(A, B):
    m = len(A)
    p = len(A[0])
    p2 = len(B)
    n = len(B[0])
    if p != p2:
        raise ValueError("dimensi tidak cocok")

    C = [[0 for j in range(n)] for i in range(m)]
    for i in range(m):
        for j in range(n):
            s = 0
            for k in range(p):
                s += A[i][k] * B[k][j]
            C[i][j] = s
    return C
```

Fungsi kerja dari kode perkalian matriks

Kita hitung dari struktur kode, bukan dari nama algoritmanya.

Bagian kode	Banyak kerja pokok
`m=len(A), p=len(A[0]), p2=len(B), n=len(B[0])`	4
`if p != p2`	1 perbandingan
Membuat matriks `C` berisi nol	mn penulisan
`s = 0` untuk setiap pasangan (i, j)	mn penulisan
`s += A[i][k] * B[k][j]`	mnp kali; tiap kali 1 kali dan 1 tambah
`C[i][j] = s`	mn penulisan

Fungsi kerja dari kode perkalian matriks

Kita hitung dari struktur kode, bukan dari nama algoritmanya.

Bagian kode	Banyak kerja pokok
`m=len(A), p=len(A[0]), p2=len(B), n=len(B[0])`	4
`if p != p2`	1 perbandingan
Membuat matriks `C` berisi nol	mn penulisan
`s = 0` untuk setiap pasangan (i, j)	mn penulisan
`s += A[i][k] * B[k][j]`	mnp kali; tiap kali 1 kali dan 1 tambah
`C[i][j] = s`	mn penulisan

$$W(m, p, n) = 5 + mn + mn + 2mnp + mn = 5 + 3mn + 2mnp.$$

Jika matriks bujur sangkar $n \times n$:

$$T(n) = 5 + 3n^2 + 2n^3 \quad \Rightarrow \quad T(n) \in O(n^3).$$

Transpose: algoritma sederhana, biaya linear terhadap jumlah elemen

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \quad \Rightarrow \quad A^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

Pseudocode

```
INPUT: A berukuran m x n
buat T berukuran n x m
FOR i = 0..m-1:
  FOR j = 0..n-1:
    T[j][i] = A[i][j]
RETURN T
```

Transpose: algoritma sederhana, biaya linear terhadap jumlah elemen

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \quad \Rightarrow \quad A^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

Pseudocode

```
INPUT: A berukuran m x n
buat T berukuran n x m
FOR i = 0..m-1:
  FOR j = 0..n-1:
    T[j][i] = A[i][j]
RETURN T
```

```python
def transpose(A):
    m = len(A)
    n = len(A[0])
    T = [[0 for i in range(m)] for j in range(n)]
    for i in range(m):
        for j in range(n):
            T[j][i] = A[i][j]
    return T

# NumPy:
# np.array(A).T
```

Fungsi kerja

Ada mn elemen yang dipindahkan. Maka $T(m, n)$ sebanding dengan mn.

Invers 2x2: rumus, domain, dan pemeriksaan

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \qquad A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

Syarat

Invers ada jika dan hanya jika

$$ad - bc \neq 0.$$

Jika determinan nol, program harus menolak input.

```
def inv2(A):
    a, b = A[0]
    c, d = A[1]
    det = a*d - b*c
    if det == 0:
        raise ValueError("matriks
            singular")
    return [[ d/det, -b/det],
            [-c/det, a/det]]
```

Invers 2x2: rumus, domain, dan pemeriksaan

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \qquad A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

```
def inv2(A):
    a, b = A[0]
    c, d = A[1]
    det = a*d - b*c
    if det == 0:
        raise ValueError("matriks
            singular")
    return [[ d/det, -b/det],
            [-c/det, a/det]]
```

Syarat

Invers ada jika dan hanya jika

$$ad - bc \neq 0.$$

Jika determinan nol, program harus menolak input.

Verifikasi

Hasil invers tidak cukup dilihat dari bentuknya. Periksa

$$AA^{-1} \approx I.$$

Pada komputer, gunakan toleransi, bukan kesamaan desimal mentah.

SPL: selesaikan, lalu ukur residual

Untuk $Ax = b$, solusi numerik $\hat{x}$ sebaiknya diperiksa dengan residual:

$$r = b - A\hat{x}.$$

Jika $\|r\|_2$ kecil, solusi konsisten dengan sistem.

Mengapa residual?

Kesalahan kecil pada komponen x dapat sulit dilihat langsung, tetapi $A\hat{x}$ harus tetap dekat dengan b.

```python
import numpy as np

A = np.array([[2, 1], [5, 3]], dtype=float)
b = np.array([1, 4], dtype=float)

x = np.linalg.solve(A, b)
r = b - A @ x
err = np.linalg.norm(r)
print(x, err)
```

SPL: selesaikan, lalu ukur residual

Untuk $Ax = b$, solusi numerik $\hat{x}$ sebaiknya diperiksa dengan residual:

$$r = b - A\hat{x}.$$

Jika $\|r\|_2$ kecil, solusi konsisten dengan sistem.

Mengapa residual?

Kesalahan kecil pada komponen x dapat sulit dilihat langsung, tetapi $A\hat{x}$ harus tetap dekat dengan b.

```python
import numpy as np

A = np.array([[2, 1], [5, 3]], dtype=float)
b = np.array([1, 4], dtype=float)

x = np.linalg.solve(A, b)
r = b - A @ x
err = np.linalg.norm(r)
print(x, err)
```

Praktik komputasi

Untuk menyelesaikan $Ax = b$, gunakan `solve`. Jangan membentuk A^{-1} hanya untuk mengalikan $A^{-1}b$.

Pengayaan: invers ukuran $n \times n$ dan biaya komputasi

Satu cara berpikir

Untuk matriks besar, invers dapat dipandang sebagai menyelesaikan

$$AX = I.$$

Kolom ke-j dari X diperoleh dari

$$Ax_j = e_j.$$

Artinya, menghitung invers berarti menyelesaikan banyak sistem linear.

Pengayaan: invers ukuran $n \times n$ dan biaya komputasi

Satu cara berpikir

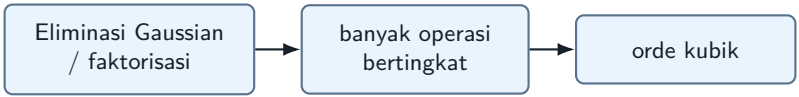
Untuk matriks besar, invers dapat dipandang sebagai menyelesaikan

$$AX = I.$$

Kolom ke-j dari X diperoleh dari

$$Ax_j = e_j.$$

Artinya, menghitung invers berarti menyelesaikan banyak sistem linear.



$$T(n) \in O(n^3), \qquad \text{memori matriks } \in O(n^2).$$

Kaitannya dengan data besar

Jika n ribuan atau lebih, membentuk invers eksplisit sering mahal dan tidak stabil. Pilih algoritma yang langsung menjawab kebutuhan: solve, least squares, sparse solver, atau metode iteratif.

Eigenvalue: lebih dari sekadar menghitung akar

Definisi

$$Av = \lambda v, \qquad v \neq 0.$$

Vektor v hanya berubah skala oleh transformasi A.

Persamaan karakteristik

$$\det(A - \lambda I) = 0.$$

Untuk matriks kecil, ini masih dapat dihitung manual.

Teorema yang berguna

Untuk matriks bujur sangkar A:

- A invertible $\iff \det(A) \neq 0$.
- A invertible $\iff 0$ bukan eigenvalue A.
- Jika eigenvalue A adalah λ_i, maka

$$\det(A) = \prod_i \lambda_i.$$

- Jika A invertible, eigenvalue A^{-1} adalah $1/\lambda_i$.

Eigen di Python: hitung dan verifikasi

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

$$\det(A - \lambda I) = \det \begin{pmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{pmatrix}$$

$$= (2-\lambda)^2 - 1 = 0$$

$$\lambda = 1, 3.$$

```python
import numpy as np

A = np.array([[2, 1], [1, 2]], dtype=float)
w, V = np.linalg.eig(A)

for k in range(len(w)):
    lam = w[k]
    v = V[:, k]
    cek = np.linalg.norm(A @ v - lam * v)
    print(lam, cek)
```

Eigen di Python: hitung dan verifikasi

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

$$\det(A - \lambda I) = \det \begin{pmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{pmatrix}$$

$$= (2-\lambda)^2 - 1 = 0$$

$$\lambda = 1, 3.$$

```python
import numpy as np

A = np.array([[2, 1], [1, 2]], dtype=float)
w, V = np.linalg.eig(A)

for k in range(len(w)):
    lam = w[k]
    v = V[:, k]
    cek = np.linalg.norm(A @ v - lam * v)
    print(lam, cek)
```

Makna pemeriksaan

Jika $\|Av - \lambda v\|_2$ kecil, pasangan (λ, v) konsisten secara numerik.

Masalah pada data berdimensi tinggi

Dalam analisis data, satu objek dapat dijelaskan oleh banyak variabel.

Misalnya suatu dataset memiliki

$$20,\ 100,\ \text{atau bahkan ribuan fitur.}$$

Semakin banyak fitur, semakin sulit untuk:

- melihat pola data;
- membandingkan antarobservasi;
- mengetahui fitur mana yang benar-benar membawa informasi baru;
- melakukan komputasi secara efisien.

Principal Component Analysis

Masalah pada data berdimensi tinggi

Dalam analisis data, satu objek dapat dijelaskan oleh banyak variabel.

Misalnya suatu dataset memiliki

$$20,\ 100,\ \text{atau bahkan ribuan fitur.}$$

Semakin banyak fitur, semakin sulit untuk:

- melihat pola data;
- membandingkan antarobservasi;
- mengetahui fitur mana yang benar-benar membawa informasi baru;
- melakukan komputasi secara efisien.

Masalah lain: fitur dapat saling berkaitan

Dua atau lebih fitur dapat berubah dengan pola yang hampir sama.
Artinya, sebagian informasi yang kita simpan sebenarnya **berulang atau saling tumpang tindih**.

Principal Component Analysis

Masalah pada data berdimensi tinggi

Dalam analisis data, satu objek dapat dijelaskan oleh banyak variabel.

Misalnya suatu dataset memiliki

$$20,\ 100,\ \text{atau bahkan ribuan fitur.}$$

Semakin banyak fitur, semakin sulit untuk:

- melihat pola data;
- membandingkan antarobservasi;
- mengetahui fitur mana yang benar-benar membawa informasi baru;
- melakukan komputasi secara efisien.

Masalah lain: fitur dapat saling berkaitan

Dua atau lebih fitur dapat berubah dengan pola yang hampir sama.
Artinya, sebagian informasi yang kita simpan sebenarnya **berulang atau saling tumpang tindih**.

Bisakah banyak fitur tersebut diringkas menjadi lebih sedikit fitur tanpa kehilangan terlalu banyak informasi?

Principal Component Analysis

PCA adalah metode untuk **meringkas data berdimensi tinggi** dengan membentuk sejumlah variabel baru yang disebut **principal components**.

Principal Component Analysis

PCA adalah metode untuk **meringkas data berdimensi tinggi** dengan membentuk sejumlah variabel baru yang disebut **principal components**.

$$\underbrace{x_1 \quad x_2 \quad x_3 \quad x_4 \quad \cdots \quad x_p}_{\text{banyak fitur}} \longrightarrow \underbrace{z_1 \quad z_2 \quad \cdots \quad z_k}_{\text{lebih sedikit komponen}}, \qquad k < p.$$

Principal Component Analysis

PCA adalah metode untuk **meringkas data berdimensi tinggi** dengan membentuk sejumlah variabel baru yang disebut **principal components**.

$$\underbrace{x_1 \quad x_2 \quad x_3 \quad x_4 \quad \cdots \quad x_p}_{\text{banyak fitur}} \longrightarrow \underbrace{z_1 \quad z_2 \quad \cdots \quad z_k}_{\text{lebih sedikit komponen}}, \qquad k < p.$$

Apa yang dilakukan PCA?

footnotesize PCA tidak sekadar membuang beberapa kolom. PCA membuat **kombinasi baru dari fitur-fitur lama** sehingga komponen pertama mempertahankan variasi data sebanyak mungkin, kemudian komponen berikutnya mempertahankan variasi yang masih tersisa.

Principal Component Analysis

PCA adalah metode untuk **meringkas data berdimensi tinggi** dengan membentuk sejumlah variabel baru yang disebut **principal components**.

$$\underbrace{x_1 \quad x_2 \quad x_3 \quad x_4 \quad \cdots \quad x_p}_{\text{banyak fitur}} \longrightarrow \underbrace{z_1 \quad z_2 \quad \cdots \quad z_k}_{\text{lebih sedikit komponen}}, \qquad k < p.$$

Apa yang dilakukan PCA?

footnotesize PCA tidak sekadar membuang beberapa kolom. PCA membuat **kombinasi baru dari fitur-fitur lama** sehingga komponen pertama mempertahankan variasi data sebanyak mungkin, kemudian komponen berikutnya mempertahankan variasi yang masih tersisa.

Hubungannya dengan aljabar linear

Principal component adalah sebuah **arah**. Karena arah dapat dinyatakan sebagai vektor, masalah PCA akhirnya menjadi masalah mencari **vektor-vektor tertentu dari sebuah matriks**.

Bagaimana PCA bekerja?

Misalkan data memiliki m observasi dan p fitur:

$$X \in \mathbb{R}^{m \times p}.$$

1. **Pusatkan data**

   Hitung rata-rata setiap fitur:

   $$\bar{x}_j = \frac{1}{m} \sum_{i=1}^{m} X_{ij},$$

   kemudian

   $$X_c = X - \mathbf{1}\bar{x}^T.$$

Bagaimana PCA bekerja?

Misalkan data memiliki m observasi dan p fitur:

$$X \in \mathbb{R}^{m \times p}.$$

1. **Pusatkan data**

   Hitung rata-rata setiap fitur:

   $$\bar{x}_j = \frac{1}{m} \sum_{i=1}^{m} X_{ij},$$

   kemudian

   $$X_c = X - \mathbf{1}\bar{x}^T.$$

2. **Hitung matriks kovariansi**

   $$S = \frac{1}{m-1} X_c^T X_c.$$

Bagaimana PCA bekerja?

Misalkan data memiliki m observasi dan p fitur:

$$X \in \mathbb{R}^{m \times p}.$$

1. **Pusatkan data**

   Hitung rata-rata setiap fitur:

   $$\bar{x}_j = \frac{1}{m} \sum_{i=1}^{m} X_{ij},$$

   kemudian

   $$X_c = X - \mathbf{1}\bar{x}^T.$$

2. **Hitung matriks kovariansi**

   $$S = \frac{1}{m-1} X_c^T X_c.$$

3. **Cari eigenvalue dan eigenvector**

   $$Sv_i = \lambda_i v_i.$$

Bagaimana PCA bekerja?

Misalkan data memiliki m observasi dan p fitur:

$$X \in \mathbb{R}^{m \times p}.$$

1. **Pusatkan data**

   Hitung rata-rata setiap fitur:

   $$\bar{x}_j = \frac{1}{m} \sum_{i=1}^{m} X_{ij},$$

   kemudian

   $$X_c = X - \mathbf{1}\bar{x}^T.$$

2. **Hitung matriks kovariansi**

   $$S = \frac{1}{m-1} X_c^T X_c.$$

3. **Cari eigenvalue dan eigenvector**

   $$Sv_i = \lambda_i v_i.$$

4. **Ambil k arah utama**

   $$W = \begin{pmatrix} v_1 & v_2 & \cdots & v_k \end{pmatrix}.$$

   Eigenvector dengan eigenvalue terbesar menjadi arah yang dipertahankan.

Bagaimana PCA bekerja?

Misalkan data memiliki m observasi dan p fitur:

$$X \in \mathbb{R}^{m \times p}.$$

1. **Pusatkan data**

   Hitung rata-rata setiap fitur:

   $$\bar{x}_j = \frac{1}{m} \sum_{i=1}^{m} X_{ij},$$

   kemudian

   $$X_c = X - \mathbf{1}\bar{x}^T.$$

2. **Hitung matriks kovariansi**

   $$S = \frac{1}{m-1} X_c^T X_c.$$

3. **Cari eigenvalue dan eigenvector**

   $$Sv_i = \lambda_i v_i.$$

4. **Ambil k arah utama**

   $$W = \begin{pmatrix} v_1 & v_2 & \cdots & v_k \end{pmatrix}.$$

   Eigenvector dengan eigenvalue terbesar menjadi arah yang dipertahankan.

5. **Proyeksikan data**

   $$Z = X_c W,$$

   dengan

   $$Z \in \mathbb{R}^{m \times k}.$$

Hasil

footnotesize Data semula memiliki p fitur.
Setelah PCA: $p \longrightarrow k, \qquad k < p.$

Algoritma data sebagai matriks

Pseudocode

```
INPUT: X berukuran m x n
hitung mean setiap kolom
buat Xc = X - mean_kolom
hitung G = Xc^T Xc
cek G simetris
ambil eigenvalue/eigenvector G jika perlu
```

Algoritma data sebagai matriks

Pseudocode

```
INPUT: X berukuran m x n
hitung mean setiap kolom
buat Xc = X - mean_kolom
hitung G = Xc^T Xc
cek G simetris
ambil eigenvalue/eigenvector G jika perlu
```

```python
import numpy as np

X = np.array([[1,2,0], [2,1,1],
              [4,3,2], [3,4,1]], dtype=float)

xbar = X.mean(axis=0)
Xc = X - xbar
G = Xc.T @ Xc
print(G)
print(np.allclose(G, G.T))
```

Fungsi kerja

Menghitung $X_c^T X_c$ untuk X_c berukuran $m \times n$ memerlukan pola seperti perkalian matriks: $n \times m$ dikali $m \times n$, orde utamanya $O(mn^2)$.

Latihan penutup yang layak menjadi soal ujian

Diberikan

$$A = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 4 \\ 2 \end{pmatrix}.$$

Kerjakan berurutan:

1. tulis pseudocode perkalian matriks-vektor Ax;
2. tulis fungsi Python untuk menghitung Ax tanpa NumPy;
3. hitung fungsi kerja untuk kode tersebut jika A berukuran $n \times n$;
4. selesaikan $Ax = b$ dengan `np.linalg.solve`;
5. hitung residual $r = b - A\hat{x}$ dan $\|r\|_2$;
6. jelaskan mengapa memeriksa residual lebih aman daripada hanya melihat nilai $\hat{x}$.

Target berpikir

Bukan sekadar memakai fungsi pustaka, tetapi memahami apa yang dihitung, bagaimana mengecek hasilnya, dan bagaimana biayanya bertumbuh.